IMPACTHUB

CS23333 –OBJECT ORIENTED PROGRAMMING USING JAVA

AND

CS23332 DATABASE MANAGEMENT SYSTEM

Submitted by

NIDHARSHA R - 2116251001163

NITHYA SREE S K - 2116251001174

PAVITHRA R R - 2116251001179

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)



RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project report titled “ImpactHub” is the bonafide work of:

NIDHARSHA R - 2116251001163

NITHYA SREE S K - 2116251001174

PAVITHRA R R - 2116251001179

who carried out the project work under the supervision of the course in charge during the academic year 2026-2027,

SIGNATURE	SIGNATURE
Dr.Neelaveni P	Mrs.Janani M
_____	_____
HEAD OF THE DEPARTMENT Information Technology Rajalakshmi Engineering College Thandalam, Chennai – 602 105	COURSE INCHARGE Information Technology Rajalakshmi Engineering College Thandalam, Chennai – 602 105

This project report is submitted for CS23333- Object Oriented Programming Using Java and CS23332- Database Management System held on _____.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ABSTRACT

Non-governmental organisations (NGOs) depend on volunteers to support community activities such as food distribution, environmental campaigns, educational support and other social-service programmes. Managing volunteer details and activity registrations manually can lead to misplaced records, duplicate entries and difficulty in tracking participation.

The NGO Volunteer Registration System is a simple terminal-based application developed using Java, JDBC and MySQL. It helps an NGO maintain volunteer information, create and view activities, register volunteers for activities, view registrations and cancel registrations. The system stores information in three related database tables: Volunteers, Activities and Registrations.

The project uses beginner-friendly Java concepts such as Scanner, loops, switch-case statements, methods and JDBC Statement. MySQL is used for storing and retrieving records. The application aims to reduce manual record-keeping and make basic volunteer coordination more organised.

TABLE OF CONTENTS

CHAPTER	CONTENTS	PAGE NO.
1	INTRODUCTION	5
	1.1 PROBLEM STATEMENT	5
	1.2 PROJECT OVERVIEW	5
	1.3 PURPOSE	5
	1.4 SCOPE	5
2	FUNCTIONALITIES PROVIDED	6
3	SYSTEM FLOW DIAGRAM	7
	3.1 SYSTEM FLOW	7
	3.2 DATA FLOW DIAGRAM	7
	3.3 USE CASE DESCRIPTION	7
	3.4 ENTITY RELATIONSHIP DESCRIPTION	8
4	MODULE DESCRIPTION	19
5	IMPLEMENTATION	10
	5.1 DATABASE CREATION	10
	5.2 JDBC CONNECTION	10
	5.3 MAIN MENU	11
	5.4 VOLUNTEER MODULE	12
	5.5 ACTIVITY MODULE	12
	5.6 REGISTRATION MODULE	13
6	OUTPUT SCREENSHOTS	14
7	CONCLUSION	17
8	REFRENCES	18

1 – INTRODUCTION

1.1 PROBLEM STATEMENT

NGOs often conduct several community-service activities and depend on volunteers to carry them out. When volunteer details and activity registrations are maintained manually, it may become difficult to find information, keep records organised and identify which volunteers have registered for particular activities. A simple computerised system is therefore useful for maintaining these records.

1.2 PROJECT OVERVIEW

The NGO Volunteer Registration System is a terminal-based Java application connected to a MySQL database through JDBC. It provides options to register volunteers, view volunteer details, add activities, view activities, register volunteers for activities, view registrations and cancel registrations. The database contains three tables connected through primary-key and foreign-key relationships.

1.3 PURPOSE

The purpose of this project is to provide a simple and organised method for maintaining NGO volunteer and activity information. It aims to reduce paper-based record keeping, make stored information easier to retrieve and support basic coordination between volunteers and NGO activities.

1.4 SCOPE

The system is designed for basic record management in a small NGO or community group. It stores volunteer contact details, activity names and locations, and the registrations made by volunteers. The project is limited to terminal-based operations and does not include online registration, payment processing, automated volunteer allocation, SMS or email notifications, or a graphical user interface.

2 – FUNCTIONALITIES PROVIDED

The following functionalities are provided by the NGO Volunteer Registration System:

- Register a new volunteer by entering name, phone number and email address.
- View the list of registered volunteers.
- Add a new NGO activity with its name and location.
- View the activities stored in the database.
- Register a volunteer for an available activity.
- View volunteer activity registrations.
- Cancel an existing registration.
- Exit the application.

The system uses simple menu-driven operations. The user selects an option from the terminal menu and enters the required information.

3 – SYSTEM FLOW DIAGRAM

The system flow describes the sequence of operations performed by the user. The application starts by establishing a database connection and displaying the main menu. Based on the selected option, the corresponding database operation is performed.

3.1 SYSTEM FLOW

STEP	PROCESS
1	Start the Java application.
2	Connect to the MySQL database.
3	Display the NGO Volunteer Registration menu.
4	Accept the user's menu choice.
5	Perform the selected operation: volunteer, activity or registration management.
6	Display the result and return to the main menu.
7	Close the application when Exit is selected.

3.2 DATA FLOW DIAGRAM

At the highest level, the user provides volunteer, activity and registration details to the system. The system stores these details in the corresponding MySQL tables and retrieves them when the user selects a view option.

EXTERNAL ENTITY	DATA SENT TO SYSTEM	DATA STORED / RETURNED
NGO Staff / User	Volunteer details	Volunteers table
NGO Staff / User	Activity details	Activities table
NGO Staff / User	Volunteer-activity registration	Registrations table
NGO Staff / User	View request	Records displayed in terminal

3.3 USE CASE DESCRIPTION

Actor: NGO Staff / Administrator

USE CASE	DESCRIPTION
Register Volunteer	Allows the user to add a volunteer's name, phone number and email address.
View Volunteers	Displays the volunteer records stored in the database.
Add Activity	Allows the user to add an NGO activity and its location.
View Activities	Displays the available activities.
Register for Activity	Stores a registration linking a volunteer with an activity.
View Registrations	Displays the registration records.
Cancel Registration	Deletes a selected registration record.
Exit	Closes the application.

3.4 ENTITY RELATIONSHIP DESCRIPTION

The database contains three entities: Volunteers, Activities and Registrations. One volunteer can have multiple registrations, and one activity can have multiple registrations. The Registrations table connects Volunteers and Activities using foreign keys.

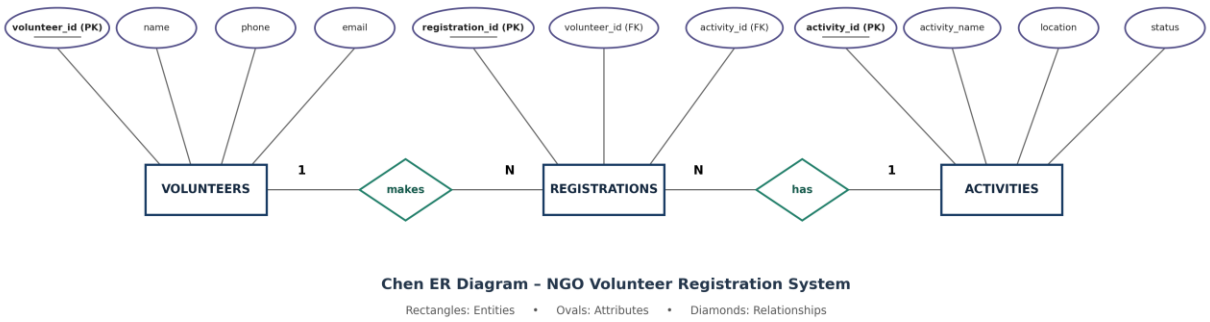


Figure 3.1: Entity Relationship Diagram (Chen Notation)

ENTITY	PRIMARY KEY	FOREIGN KEY / RELATION
Volunteers	volunteer_id	Parent entity for Registrations
Activities	activity_id	Parent entity for Registrations
Registrations	registration_id	volunteer_id references Volunteers; activity_id references Activities

4 – MODULE DESCRIPTION

VOLUNTEER MODULE

This module allows the NGO staff to register new volunteers by entering their name, phone number and email address. It also provides an option to view all registered volunteers.

ACTIVITY MODULE

This module allows the user to add activities conducted by the NGO. Each activity contains an activity name and location. The user can also view the activities stored in the database.

REGISTRATION MODULE

This module connects a volunteer with an activity. The user enters the volunteer ID and activity ID to create a registration. Existing registrations can be viewed, and a registration can be cancelled using its registration ID.

DATABASE MODULE

The database module stores all project information in MySQL. The three tables are related using primary keys and foreign keys. Java communicates with the database through JDBC.

5 – IMPLEMENTATION

5.1 DATABASE CREATION

The database is created in MySQL using the name ImpactHub. The following SQL statements create the three required tables.

```
CREATE DATABASE ngo_management;
USE ngo_management;

CREATE TABLE Volunteers (
    volunteer_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50),
    phone VARCHAR(15),
    email VARCHAR(100)
);

CREATE TABLE Activities (
    activity_id INT PRIMARY KEY AUTO_INCREMENT,
    activity_name VARCHAR(100),
    location VARCHAR(100),
    status VARCHAR(20)
);

CREATE TABLE Registrations (
    registration_id INT PRIMARY KEY AUTO_INCREMENT,
    volunteer_id INT,
    activity_id INT,
    FOREIGN KEY (volunteer_id) REFERENCES Volunteers(volunteer_id),
    FOREIGN KEY (activity_id) REFERENCES Activities(activity_id)
);
```

The volunteer_id and activity_id columns in Registrations are foreign keys. They ensure that a registration refers to an existing volunteer and an existing activity.

5.2 JDBC CONNECTION

JDBC (Java Database Connectivity) is used to connect the Java application with the MySQL database. The Connection object represents the database connection, and Statement is used to execute basic SQL queries. Replace the password placeholder with the password configured for the local MySQL account.

```
import java.sql.*;

public class DBConnection {
    public static Connection getConnection() {
        Connection con = null;

        try {
            con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/ImpactHub",
                "root",
                "YOUR_PASSWORD"
            );
        } catch (Exception e) {
            System.out.println(e);
        }

        return con;
```

```
    }
}
```

MySQL Connector/J must be added to the Java project's classpath. The connection class does not print a database-connected success message.

5.3 MAIN MENU

The main menu is displayed in the terminal. A while loop keeps the menu running until the user selects Exit. A switch-case statement calls the operation selected by the user.

```
===== IMPACTHUB =====
1. Register Volunteer
2. View Volunteers
3. Add Activity
4. View Activities
5. Register for Activity
6. View Registrations
7. Cancel Registration
8. Exit

Enter your choice:
```

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int choice = 0;

        while (choice != 8) {
            System.out.println("1. Register Volunteer");
            System.out.println("2. View Volunteers");
            System.out.println("3. Add Activity");
            System.out.println("4. View Activities");
            System.out.println("5. Register for Activity");
            System.out.println("6. View Registrations");
            System.out.println("7. Cancel Registration");
            System.out.println("8. Exit");

            System.out.print("Enter your choice: ");
            choice = sc.nextInt();
            sc.nextLine();

            switch (choice) {
                case 1: // Register volunteer
                    break;
                case 2: // View volunteers
                    break;
                case 3: // Add activity
                    break;
                case 4: // View activities
                    break;
                case 5: // Register for activity
                    break;
                case 6: // View registrations
                    break;
                case 7: // Cancel registration
                    break;
                case 8:
                    System.out.println("Exiting...");
                    break;
                default:
```

```
                    System.out.println("Invalid choice");
            }
        }
        sc.close();
    }
}
```

The above menu code shows the basic control structure. The comments are placeholders for the database operation methods implemented in the final application.

5.4 VOLUNTEER MODULE

Register Volunteer: The user enters the volunteer's name, phone number and email address. The system inserts the details into the Volunteers table.

```
String query = "INSERT INTO Volunteers(name, phone, email) "
             + "VALUES('" + name + "', '" + phone + "', '" + email + "')";
Statement st = con.createStatement();
st.executeUpdate(query);
```

View Volunteers: The system retrieves all volunteer records and displays them in the terminal.

```
String query = "SELECT * FROM Volunteers";
Statement st = con.createStatement();
ResultSet rs = st.executeQuery(query);

while (rs.next()) {
    System.out.println(rs.getInt("volunteer_id") + " "
        + rs.getString("name") + " "
        + rs.getString("phone") + " "
        + rs.getString("email"));
}
```

5.5 ACTIVITY MODULE

Add Activity: The user enters the activity name and location. The activity is inserted into the Activities table. The status can initially be stored as Open.

```
String query = "INSERT INTO Activities(activity_name, location, status) "
             + "VALUES('" + activityName + "', '" + location + "', 'Open')";
Statement st = con.createStatement();
st.executeUpdate(query);
```

View Activities: The system retrieves and displays the activity ID, activity name, location and status.

```
String query = "SELECT * FROM Activities";
Statement st = con.createStatement();
ResultSet rs = st.executeQuery(query);

while (rs.next()) {
    System.out.println(rs.getInt("activity_id") + " "
        + rs.getString("activity_name") + " "
        + rs.getString("location") + " "
        + rs.getString("status"));
}
```

5.6 REGISTRATION MODULE

Register for Activity: The user enters an existing volunteer ID and activity ID. The system inserts these values into the Registrations table. The foreign keys ensure that both referenced records exist.

```
String query = "INSERT INTO Registrations(volunteer_id, activity_id) "
             + "VALUES(" + volunteerId + ", " + activityId + ")";
Statement st = con.createStatement();
st.executeUpdate(query);
```

View Registrations: The system retrieves registration records and displays their IDs, volunteer IDs and activity IDs.

```
String query = "SELECT * FROM Registrations";
Statement st = con.createStatement();
ResultSet rs = st.executeQuery(query);

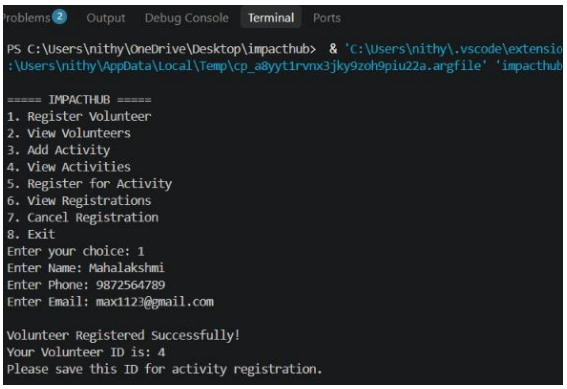
while (rs.next()) {
    System.out.println(rs.getInt("registration_id") + " "
        + rs.getInt("volunteer_id") + " "
        + rs.getInt("activity_id"));
}
```

Cancel Registration: The user enters a registration ID. The selected registration is removed from the Registrations table.

```
String query = "DELETE FROM Registrations WHERE registration_id = "
             + registrationId;
Statement st = con.createStatement();
st.executeUpdate(query);
```

6.OUTPUT SCREENSHOTS

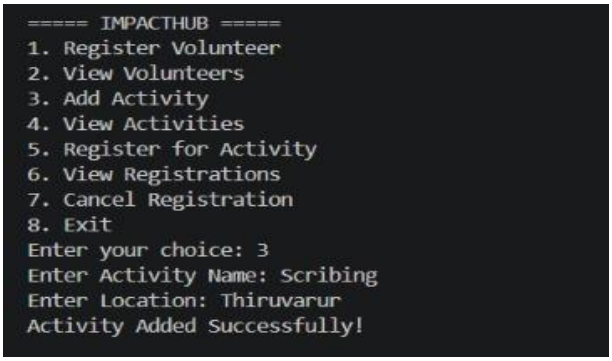
6.1 CHOICE 1



6.2 CHOICE 2



6.3 CHOICE 3



6.4 CHOICE 4

```
===== IMPACTHUB =====
1. Register Volunteer
2. View Volunteers
3. Add Activity
4. View Activities
5. Register for Activity
6. View Registrations
7. Cancel Registration
8. Exit
Enter your choice: 4
Activity ID: 1
Activity: Tree Plantation
Location: Chennai
Status: Open
--------------------
Activity ID: 2
Activity: Food Distribution
Location: Tambaram
Status: Open
--------------------
Activity ID: 3
Activity: Teaching Children
Location: Velachery
Status: Open
--------------------
Activity ID: 4
Activity: Beach Cleanup
Location: Marina Beach
Status: Open
--------------------
Activity ID: 5
Activity: Blood Donation Camp
Location: Guindy
Status: Open
--------------------
```

6.5 CHOICE 3

```
===== IMPACTHUB =====
1. Register Volunteer
2. View Volunteers
3. Add Activity
4. View Activities
5. Register for Activity
6. View Registrations
7. Cancel Registration
8. Exit
Enter your choice: 5
Enter Volunteer ID: 3
Enter Activity ID: 6
Registered for Activity Successfully!
```

6.6 CHOICE 6

```
===== IMPACTHUB =====
1. Register Volunteer
2. View Volunteers
3. Add Activity
4. View Activities
5. Register for Activity
6. View Registrations
7. Cancel Registration
8. Exit
Enter your choice: 6
Registration ID: 1
Volunteer: Nithya Sree S k
Activity: Food Distribution
--------------------
Registration ID: 3
Volunteer: Andrea
Activity: plantation drive
--------------------
```

6.7 CHOICE 7

```
===== IMPACTHUB =====
1. Register Volunteer
2. View Volunteers
3. Add Activity
4. View Activities
5. Register for Activity
6. View Registrations
7. Cancel Registration
8. Exit
Enter your choice: 7
Enter Registration ID to cancel: 1
Registration Cancelled Successfully!
```

6.8 CHOICE 8

```
===== IMPACTHUB =====
1. Register Volunteer
2. View Volunteers
3. Add Activity
4. View Activities
5. Register for Activity
6. View Registrations
7. Cancel Registration
8. Exit
Enter your choice: 8
Thank you for using ImpactHub!
```

6.9 VOLUNTEER TABLE

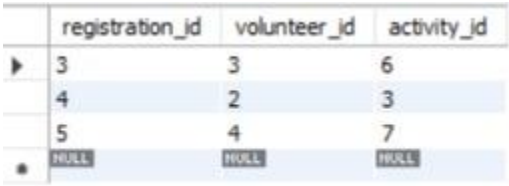
volunteer_id	name	phone	email
1	Nithya Sree S k	1234567890	abc@gamil.com
2	Nithya Sree S k	1234567890	abc@gmail.com
3	Andrea	1876492783	ceg@gmail.com
4	Mahalakshmi	9872564789	max1123@gmail.com
NULL	NULL	NULL	NULL

6.10 ACTIVITIES TABLE



activity_id	activity_name	location	status
1	Tree Plantation	Chennai	Open
2	Food Distribution	Tambaram	Open
3	Teaching Children	Velachery	Open
4	Beach Cleanup	Marina Beach	Open
5	Blood Donation Camp	Guindy	Open
6	plantation drive	chennai	Open
7	Scribing	Thiruvarur	Open
NULL	NULL	NULL	NULL

6.11 REGISTRATION TABLE



registration_id	volunteer_id	activity_id
3	3	6
4	2	3
5	4	7
NULL	NULL	NULL

7– CONCLUSION

The NGO Volunteer Registration System provides a simple way to maintain volunteer details, NGO activities and volunteer registrations. The application uses Java for the terminal-based menu and JDBC to communicate with a MySQL database. The database is organised into three related tables: Volunteers, Activities and Registrations.

The project demonstrates basic object-oriented programming concepts, menu-driven programming, SQL operations and database connectivity. It can help a small NGO maintain its records in a more organised manner than manual record keeping. Through this project, we gain practical experience in Java programming, MySQL database creation and JDBC connectivity.

The system can be extended in the future with features such as volunteer attendance, activity completion tracking, search options and reports.

8– REFERENCES

- Java Documentation – https://docs.oracle.com/en/java/
- MySQL Reference Manual – https://dev.mysql.com/doc/
- MySQL Connector/J Developer Guide – https://dev.mysql.com/doc/connector-j/en/
- Course material for IT19341 – Introduction to OOP and Java.